

Predictive Maintenance for Engine Health

Final Project Report

Generated on: 2026-05-16

Executive Summary

This project develops a machine learning solution that classifies whether an engine is operating normally or needs maintenance based on RPM, oil pressure, fuel pressure, coolant pressure, oil temperature, and coolant temperature.

The final model selected by test F1 score is AdaBoost. On the holdout test set, it achieved accuracy 65.40%, precision 65.37%, recall 95.94%, F1 score 77.76%, and ROC-AUC 68.39%.

The business value is earlier identification of likely engine issues, which can reduce unplanned breakdowns, improve maintenance scheduling, and give fleet/service teams a data-backed triage signal.

1. Business Context and Objective

Unexpected engine failure creates repair cost, operational downtime, safety risk, and poor customer experience. Predictive maintenance uses sensor readings to detect risk before failure happens.

The objective is to convert historical engine sensor data into a classification model that can support proactive maintenance decisions.

2. Data Registration

A master project folder was created with a dedicated data subfolder. The raw CSV was stored under data/raw and the processed train/test datasets were stored under data/processed.

Raw data registration status: uploaded to Hugging Face dataset repo: [prohra48/predictive-maintenance-engine-data](#).

Hugging Face dataset link: [prohra48/predictive-maintenance-engine-data](#).

3. Exploratory Data Analysis

Methodology

The dataset contains 19,535 records and 7 columns. There are 0 missing values and 0 duplicate rows after cleaning.

The target variable is moderately imbalanced: 7,218 normal records (36.95%) and 12,317 maintenance records (63.05%).

The EDA uses summary statistics, target distribution, univariate histograms, boxplots by condition, and a correlation heatmap to compare engine behavior under normal and maintenance states.

Table 1. Data Overview

Unnamed: 0	count	mean	std	min	25%	50%	75%	max
engine_rpm	19535.00	791.24	267.61	61.00	593.00	746.00	934.00	2239.00
lub_oil_pressure	19535.00	3.30	1.02	0.0000	2.52	3.16	4.06	7.27
fuel_pressure	19535.00	6.66	2.76	0.0000	4.92	6.20	7.74	21.14
coolant_pressure	19535.00	2.34	1.04	0.0000	1.60	2.17	2.85	7.48
lub_oil_temp	19535.00	77.64	3.11	71.32	75.73	76.82	78.07	89.58
coolant_temp	19535.00	78.43	6.21	61.67	73.90	78.35	82.92	195.53
engine_condition	19535.00	0.6300	0.4800	0.0000	0.0000	1.0000	1.0000	1.0000

Table 2. Target Distribution

engine_condition	records	percentage
0.0000	7218.00	36.95
1.0000	12317.00	63.05

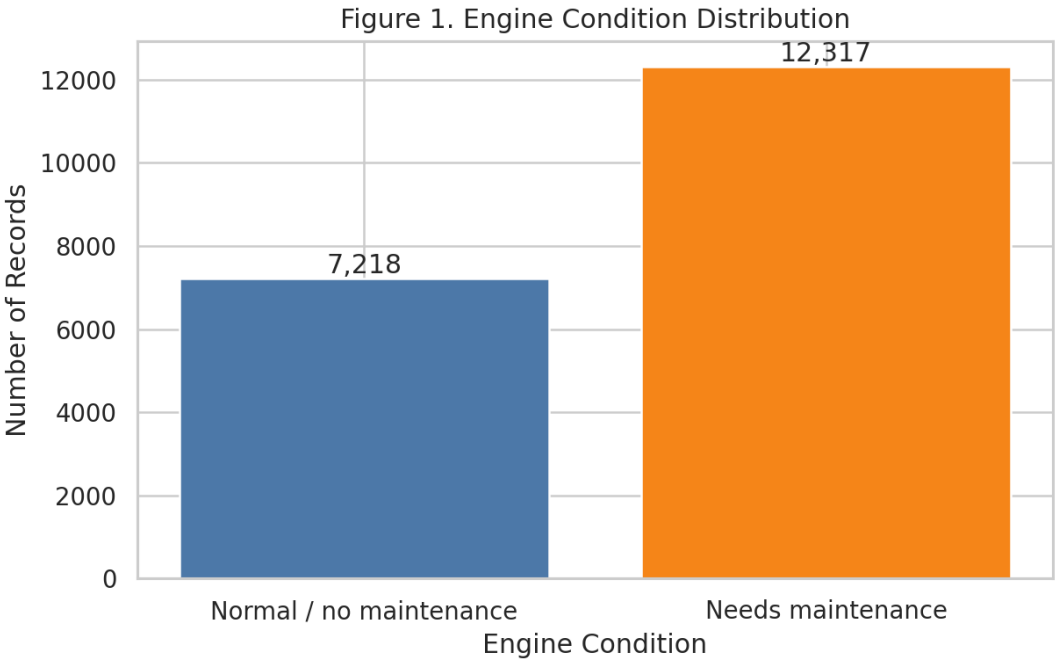


Figure 1. Engine Condition Distribution

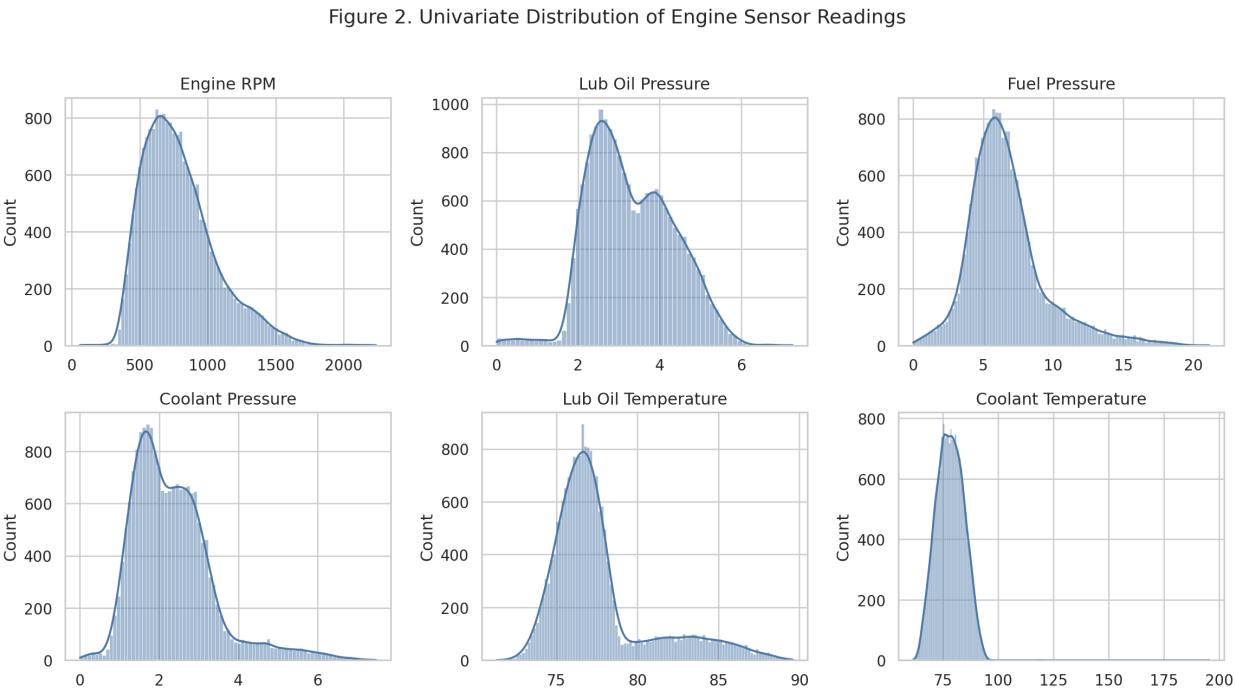


Figure 2. Univariate Distribution of Engine Sensor Readings

Figure 3. Sensor Readings by Engine Condition

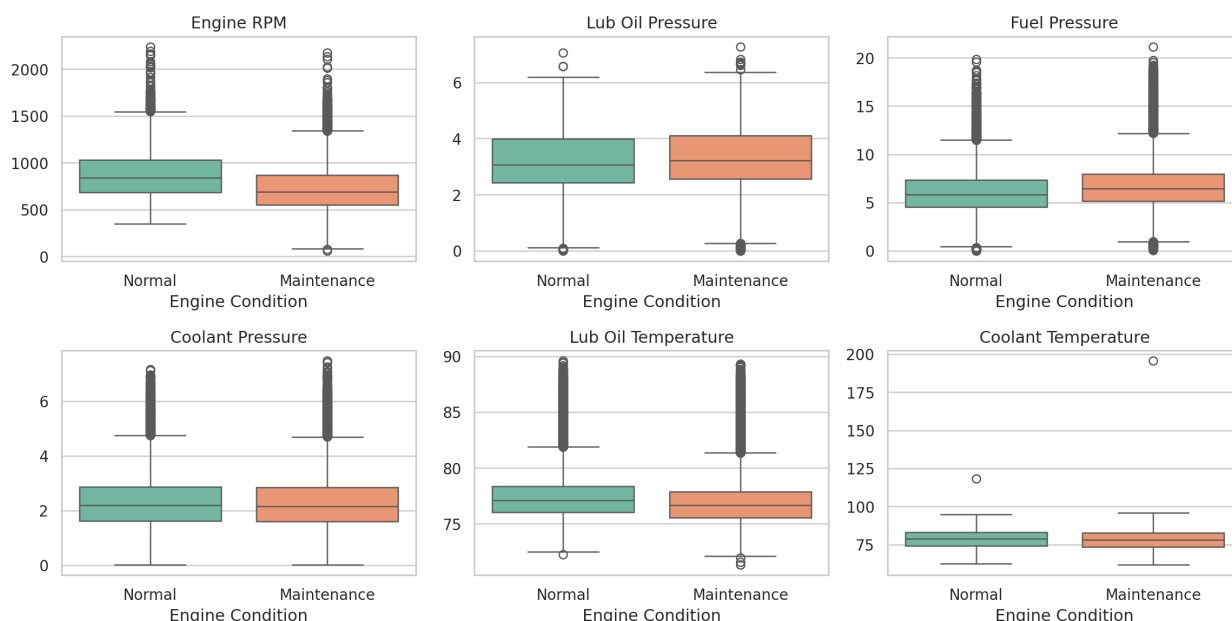


Figure 3. Sensor Readings by Engine Condition

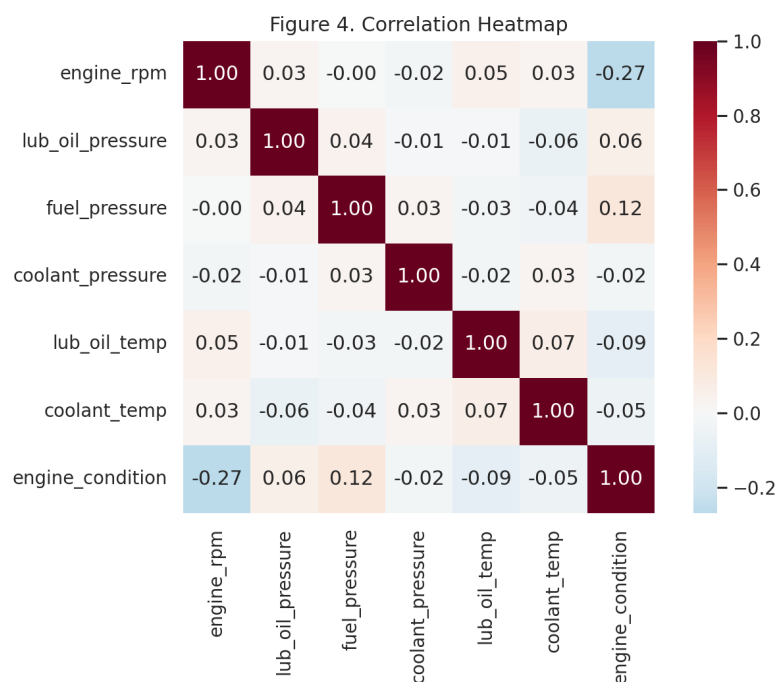


Figure 4. Correlation Heatmap

4. Data Preparation

Methodology

Column names were standardized to snake_case, numeric fields were validated, duplicate rows were checked, and no unnecessary columns were retained. After Hugging Face publication, setting USE_HF_DATA=1 makes the scripts load the raw/train/test files directly from the Hugging Face dataset repo.

The data was split into 15,628 training rows and 3,907 testing rows using a stratified split so the target distribution remains consistent.

Processed data upload status: uploaded clean/train/test datasets to Hugging Face dataset repo: [prohra48/predictive-maintenance-engine-data](#).

5. Model Building with Experimentation Tracking

Methodology

Five classification algorithms were evaluated: Decision Tree, Random Forest, Gradient Boosting, AdaBoost, and XGBoost. Hyperparameters were tuned with cross-validated grid search and the tuned parameters were logged in `experiments/experiment_tracking.csv`.

The best model is AdaBoost, selected primarily by F1 score because the business problem requires a balance between catching maintenance cases and avoiding excessive false alarms.

The strongest model signal is led by Engine Rpm, based on the saved feature-importance chart. This helps translate the model output into practical engineering signals.

Model hub status: uploaded best model to Hugging Face model repo: [prohra48/predictive-maintenance-engine-model](#).

Table 3. Model Experiment Tracking Summary

accuracy	precision	recall	f1	roc_auc	best_cv_f1	best_params
0.40	0.6537	0.9594	0.7776	0.6839	0.7743	{"model__learning_rate": 0.05, "model__n_estimators": 150}
0.60	0.6835	0.8758	0.7678	0.7003	0.7686	{"model__colsample_bytree": 0.9, "model__learning_rate": 0.05, "model__max_depth": 2, "model__n_estimators": 100,
0.47	0.6842	0.8693	0.7657	0.7004	0.7670	{"model__learning_rate": 0.05, "model__max_depth": 2, "model__n_estimators": 100}
0.22	0.6865	0.8250	0.7494	0.6808	0.7520	{"model__max_depth": null, "model__min_samples_leaf": 1, "model__n_estimators": 250}
0.95	0.6633	0.6760	0.6696	0.5454	0.6697	{"model__max_depth": null, "model__min_samples_leaf": 1, "model__min_samples_split": 2}

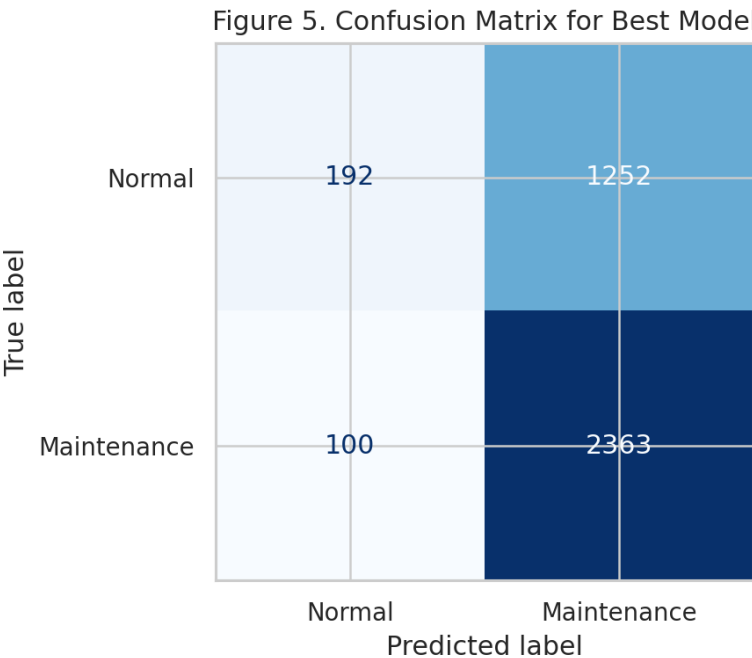


Figure 5. Confusion Matrix for Best Model

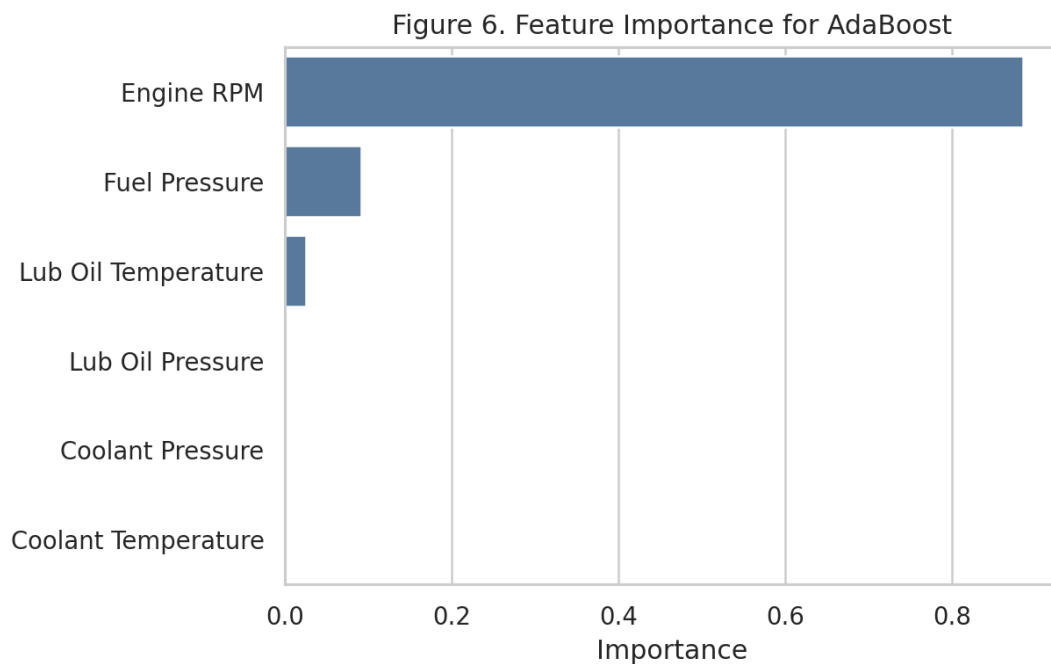


Figure 6. Feature Importance for Best Model

6. Model Deployment

A Streamlit app is provided in `deployment/app.py`. It accepts the six sensor inputs, loads the saved model, and returns the predicted engine condition with the estimated maintenance probability when available.

`deployment/requirements.txt` lists the runtime dependencies. `deployment/Dockerfile` defines a containerized configuration for reproducible hosting.

Hugging Face Space link: [prohra48/predictive-maintenance-engine-app](https://huggingface.co/prohra48/predictive-maintenance-engine-app).

7. Automated GitHub Actions Workflow

`.github/workflows/pipeline.yml` defines an automated workflow that installs dependencies, runs data registration, EDA, data preparation, model training, report generation, and artifact upload.

If Hugging Face secrets are added in GitHub, the workflow can also upload datasets/models and push the Streamlit app files to a Hugging Face Space.

Required GitHub secrets: `HF_TOKEN`, `HF_DATASET_REPO`, `HF_MODEL_REPO`, and `HF_SPACE_REPO`.

8. Output Evaluation

GitHub repository link: to be filled after pushing this project to GitHub.

GitHub Actions screenshot: insert screenshot after a successful workflow run.

Hugging Face Space screenshot: insert screenshot after the Streamlit app is published and tested.

9. Actionable Insights and Recommendations

Use the model as a triage signal rather than a full replacement for mechanical inspection.

Prioritize recall for maintenance cases when the cost of missing an engine issue is higher than the cost of an extra inspection.

Monitor model performance after deployment because operating ranges may shift across engine types, seasons, and usage patterns.

Collect service-confirmed labels over time and retrain the model periodically to keep predictions aligned with real maintenance outcomes.

Appendix A. Reproducible Local Commands

```
pip install -r requirements.txt
python src\01_data_registration.py
python src\02_eda.py
python src\03_data_preparation.py
python src\04_model_training.py
python src\05_generate_report.py
python src\06_generate_notebook.py
streamlit run deployment\app.py
```

Appendix B. Code and Evidence Files

src/01_data_registration.py

src/02_eda.py

src/03_data_preparation.py

src/04_model_training.py

src/05_generate_report.py

deployment/app.py

deployment/Dockerfile

.github/workflows/pipeline.yml

experiments/experiment_tracking.csv

models/model_metadata.json